

I used Microsoft 365 Copilot, based on the GPT-5 reasoning model to evaluate my explanations of neural networks, loss functions, and convolutional neural networks.

1. Prompt: Can you generate 3 essay questions related to deep neural networks, loss functions, and convolutional neural networks to test me.

Response:

Here are three essay questions designed to test both your conceptual understanding and your ability to connect ideas.

1. Deep Neural Networks

Explain how information moves through a deep neural network during **forward propagation** and how the network learns during **backpropagation**. Discuss the roles of weights, biases, activation functions, gradients, and the learning rate. Also explain how vanishing or exploding gradients can affect training.

2. Loss Functions

What is the purpose of a loss function when training a neural network? Compare **mean squared error** and **cross-entropy loss**, including how each is calculated, the types of problems for which each is appropriate, and what could happen if an inappropriate loss function is selected.

3. Convolutional Neural Networks

Explain how a convolutional neural network processes an image. Describe the roles of convolutional filters, feature maps, stride, padding, pooling layers, and fully connected layers. Additionally, explain why CNNs are generally more effective for image recognition than standard fully connected neural networks.

2. A. We haven't discussed forward propagation or backpropagation yet so I will ignore that question for now. Weights are used to adjust the outputs from the activation functions. Biases are also added when adding up the hidden units and inside of the activation functions. Activation functions are used to make linear functions into nonlinear behavior where these functions can represent nonlinear behavior from linear functions. Gradients are used to measure how high or low the loss produced from the models where in higher gradients the loss is minimized and closer to a perfect fit for the data. The learning rate is parameter that is used during gradient descent to specify the size of the steps taken to reach the minimum of the loss function. We have not discussed vanishing or exploding gradients yet.
B. Purpose of a loss function is to measure how close the network can guess the correct answer of a certain input. Mean squared error is used inside of 1D regression models

where you use normal distribution for the probability distribution. Cross entropy loss is what is used in models where you want more than one output. Where it can measure how close the predicted answer is to what the actual answer is.

C. A convolutional neural network processes an image by using a kernel to take in multiple inputs into one output, but that doesn't mean it's only one output produced. The kernel takes strides when processing the input so that it can "skip" over certain inputs, so it doesn't process the same nodes for one output. Padding is used when this kernel will overstep the boundaries of these layers where it will read "0" for every nonexistent node that isn't in the input. Fully connected layers are layers that have connected inputs to the outputs, where each input is connected to an output. CNNs are more effective for image recognition because they don't process every single detail about the image. Images can have a lot of information and processing every single input of an image is not efficient at all (like using fully connected neural networks). So, CNNs use only certain patterns within the image rather than try to connect every single pixel.

3. Prompt: Can you critique my answers to the set of questions:

We haven't discussed forward propagation or backpropagation yet so I will ignore that question for now. Weights are used to adjust the outputs from the activation functions. Biases are also added when adding up the hidden units and inside of the activation functions. Activation functions are used to make linear functions into nonlinear behavior where these functions can represent nonlinear behavior from linear functions. Gradients are used to measure how high or low the loss produced from the models where in higher gradients the loss is minimized and closer to a perfect fit for the data. The learning rate is parameter that is used during gradient descent to specify the size of the steps taken to reach the minimum of the loss function. We have not discussed vanishing or exploding gradients yet.

B. Purpose of a loss function is to measure how close the network can guess the correct answer of a certain input. Mean squared error is used inside of 1D regression models where you use normal distribution for the probability distribution. Cross entropy loss is what is used in models where you want more than one output. Where it can measure how close the predicted answer is to what the actual answer is.

C. A convolutional neural network processes an image by using a kernel to take in multiple inputs into one output, but that doesn't mean it's only one output produced. The kernel takes strides when processing the input so that it can "skip" over certain inputs, so it doesn't process the same nodes for one output. Padding is used when this kernel will overstep the boundaries of these layers where it will read "0" for every nonexistent node that isn't in the input. Fully connected layers are layers that have connected inputs to the outputs, where each input is connected to an output. CNNs are more effective for image recognition because they don't process every single detail about the image. Images can have a lot of information and processing every single input of an image is not efficient at all (like using fully connected neural networks). So, CNNs use only certain patterns within the image rather than try to connect every single pixel.

Response:

Your completed first answer shows a reasonable understanding of the main components of a neural network. The most important correction is your explanation of gradients: a higher gradient does not mean lower loss or a better fit.

A. Deep Neural Networks

What you explained correctly

- You recognized that **weights affect a neuron's output**.
- You correctly connected biases with the calculation performed by a neuron.
- You correctly explained that activation functions introduce **nonlinear behavior**.
- Your explanation of the learning rate as the **step size used during gradient descent** is correct.
- It is reasonable to omit backpropagation and vanishing or exploding gradients because your class has not introduced them.

Corrections

1. Weights

You wrote:

"Weights are used to adjust the outputs from the activation functions."

More accurately, weights adjust the **inputs before the activation function is applied**. Each input is multiplied by a weight, and the resulting values are added together.

A neuron generally performs:

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

Then the activation function is applied:

$$a = f(z)$$

A better explanation would be:

Weights determine how strongly each input influences a neuron. The weighted inputs are added together before being passed into the activation function.

2. Biases

You wrote:

“Biases are also added when adding up the hidden units and inside of the activation functions.”

The bias is added to the **weighted sum before** that value enters the activation function. It is not usually described as being inside the activation function.

A clearer version is:

A bias is added to the weighted sum of the inputs before the activation function is applied. It allows the neuron to shift its output rather than being restricted by the input values alone.

3. Activation functions

Your central idea is correct, but the wording can be more precise. The activation function does not turn a linear function itself into a nonlinear function. Instead, it **introduces nonlinearity into the network**.

A better version is:

Activation functions introduce nonlinear behavior into a neural network, allowing it to learn more complicated patterns than a network made entirely from linear operations.

4. Gradients

You wrote:

“Gradients are used to measure how high or low the loss produced from the models where in higher gradients the loss is minimized and closer to a perfect fit for the data.”

This is the main technical error. A gradient does not directly measure how high or low the loss is. It measures **how the loss changes when a parameter changes**.

Also, a higher gradient does not necessarily mean that the loss is minimized. A large gradient usually means the loss is changing rapidly. Near a minimum, the gradient is often close to zero.

A corrected explanation is:

Gradients measure how much and in what direction the loss changes when the network’s weights or biases change. Gradient descent uses this information to adjust the parameters in the direction that reduces the loss. Near a minimum, the gradient is usually close to zero.

5. Learning rate

Your explanation is mostly correct. A slightly more complete version is:

The learning rate is a parameter that controls the size of the steps taken during gradient descent. If it is too large, the model may pass over the minimum. If it is too small, training may take a long time.

Revised answer for A

We have not discussed forward propagation, backpropagation, or vanishing and exploding gradients, so I will focus on the other components. **Weights determine how strongly each input influences a neuron. The weighted inputs are added together with a bias before being passed into an activation function. The bias allows the neuron to shift its output. Activation functions introduce nonlinear behavior, which allows a neural network to learn complicated patterns. Gradients measure how much and in what direction the loss changes when the weights or biases change. The learning rate determines the size of the steps taken during gradient descent as the model attempts to minimize its loss.**

B. Loss Functions

What you explained correctly

- You correctly identified the main purpose of a loss function.
- You correctly associated mean squared error with regression.
- You correctly recognized that cross-entropy compares the prediction with the correct answer.

Corrections

1. Purpose of a loss function

Instead of saying it measures how closely the network can “guess,” use more precise language:

A loss function measures the difference between a model’s prediction and the correct output.

A smaller loss generally indicates that the predictions are closer to the correct values.

2. Mean squared error

You wrote that it is used in “1D regression models.” That is too restrictive. Mean squared error can be used in regression models with one or multiple predicted numerical values.

A better explanation is:

Mean squared error is commonly used for regression problems in which the model predicts continuous numerical values. It squares the difference between each predicted value and actual value and then calculates the average.

A normal-error distribution can help mathematically justify using mean squared error, but mean squared error is not limited to "1D" models.

3. Cross-entropy loss

Cross-entropy is not defined by whether the model has more than one output. It is primarily used for **classification**.

For example:

- Binary cross-entropy is commonly used for two-class classification.
- Categorical cross-entropy is commonly used when choosing among multiple classes.

A better explanation is:

Cross-entropy loss is commonly used for classification. It measures how closely the model's predicted class probabilities match the correct class. It assigns a larger penalty when the model confidently predicts the wrong class.

Revised answer for B

The purpose of a loss function is to measure the difference between a model's predicted output and the correct output. Mean squared error is commonly used for regression problems where the model predicts continuous numerical values. It calculates the average squared difference between the predicted and actual values. Cross-entropy loss is commonly used for classification and measures how closely the model's predicted probabilities match the correct class. Choosing an inappropriate loss function can cause the model to learn in a way that does not match the goal of the problem.

C. Convolutional Neural Networks

What you explained correctly

- You correctly explained that a kernel combines several input values to produce an output value.
- You recognized that the convolution produces multiple output values rather than only one.
- You understand the general purpose of stride and padding.

- You correctly recognized that fully connected networks require many connections.
- You correctly explained that CNNs look for useful patterns in an image.

Corrections

1. Kernel and feature map

A kernel moves across small regions of an image. At each position, it produces one value. All these values together create a **feature map**.

A clearer explanation is:

A kernel is a small matrix that moves across an image and performs a calculation at each position. The resulting output values form a feature map that can represent patterns such as edges, textures, or shapes.

2. Stride

You wrote that stride allows the kernel to skip inputs so that it does not process the same nodes for one output. This is partially correct, but neighboring kernel positions can still overlap.

A more accurate explanation is:

Stride determines how many positions the kernel moves after each calculation. A stride of one moves the kernel one position at a time, while a larger stride skips some positions and creates a smaller feature map.

3. Padding

Padding is intentionally added around an image before the kernel moves across it. It is not added only when a kernel accidentally oversteps the boundary.

A corrected explanation is:

Padding adds values, usually zeros, around the outside of the input. This helps the kernel process pixels near the edges and can prevent the feature map from becoming too small.

4. Fully connected layers

Your explanation is close, but it should specify that every neuron in one layer connects to every neuron in the following layer:

In a fully connected layer, every neuron from the previous layer is connected to every neuron in the next layer.

5. Why CNNs are effective

CNNs still process the image's pixels, so saying they do not process every detail could be misleading. Their advantage is that they process **small local areas** and reuse the same kernel throughout the image.

You also omitted pooling, which was part of the original question.

A better explanation is:

CNNs are effective because they examine small local regions and reuse the same kernels across the image. This allows them to recognize patterns using fewer parameters than a fully connected network. Pooling can then reduce the dimensions of the feature maps while preserving important features.

Revised answer for C

A convolutional neural network processes an image using kernels, also called filters. A kernel moves across small regions of the image and produces one value at each position. These values form a feature map that can represent patterns such as edges, textures, and shapes. Stride determines how far the kernel moves between calculations. Padding adds values, usually zeros, around the input so the kernel can process the edges and preserve the feature map's dimensions. Pooling reduces the size of feature maps while retaining important information. Fully connected layers connect every neuron in one layer to every neuron in the next layer and can use the features found by earlier layers to make a final prediction. CNNs are effective for images because they reuse kernels to recognize local patterns, requiring fewer parameters than a fully connected network.

Overall assessment

- **A:** Good beginning, but the gradient explanation needs a significant correction.
- **B:** Correct overall purpose, but use **regression versus classification** instead of **one output versus multiple outputs**.
- **C:** Strongest answer conceptually. Refine your descriptions of stride and padding, and add feature maps and pooling.

Main improvements

- **Technical accuracy:** Gradients indicate the direction and rate of change of loss, not how close the model is to a perfect fit.
- **Terminology:** Use “continuous numerical values” for regression and “class probabilities” for classification.
- **Clarity:** Explain CNN efficiency through local connections and shared kernels rather than saying CNNs do not process every detail.

4.

A. Weights help tell how strongly each input affects the neuron. These weights are then added together with the biases before being added to the activation function. The bias has the output shift its output. Activation functions show nonlinear behavior which can help neural networks learn more complex patterns. Gradients are used to determine what direction and how much the loss changes when weights and biases are changed. The learning rate changes how big the steps are when doing gradient descent when the model wants to minimize loss.

Changed the order in which weights, biases, and activation functions are used. Also changed the explanation of gradients from measuring model accuracy to measuring how the loss changes when weights and biases change.

B. The loss function measures the difference between what the models predicts to be the output and what the correct output is. Mean squared error is used for regression problems where the model wants to predict continuous values. It calculates the average of squared difference between the predicted values and the correct values. Cross entropy loss is used for classification and measures how close the model’s predicted probabilities for the correct class are. When an inappropriate loss function is used, it can cause the model to learn in a different way than what is needed for the problem.

Clarified that mean squared error is used for regression with continuous values, rather than 1D regression. Also changed cross entropy from being associated with multiple outputs to being associated with classification and predicted probabilities.

C. A convolutional neural network processed an image by using filters, kernels. A kernel moves across regions of an image to produce a value for each position. These values form a feature map that can represent patterns like edges, textures, and shapes. The stride determines how much the kernel moves between calculations. Padding adds values (typically zeros) around the input so the kernel can process the edges and keep the feature maps dimensions. Pooling reduces the size of feature maps without discarding the important features. Fully connected layers connect every neuron in one layer to every neuron in the following layer so they use information from the previous layers to output a prediction. CNNs are effective for images because they can reuse kernels to find patterns which requires fewer parameters than a fully connected network.

Added missing concepts such as feature maps and pooling while making explanations about stride and padding more accurate. Improved the explanation of CNN efficiency by focusing on kernels, detecting local patterns, and requiring fewer parameters than fully connected networks.

Prompt:

Can you find any gaps inside of this explanation to the previous questions?

A. Weights help tell how strongly each input affects the neuron. These weights are then added together with the biases before being added to the activation function. The bias has the output shift its output. Activation functions show nonlinear behavior which can help neural networks learn more complex patterns. Gradients are used to determine what direction and how much the loss changes when weights and biases are changed. The learning rate changes how big the steps are when doing gradient descent when the model wants to minimize loss.

B. The loss function measures the difference between what the models predicts to be the output and what the correct output is. Mean squared error is used for regression problems where the model wants to predict continuous values. It calculates the average of squared difference between the predicted values and the correct values. Cross entropy loss is used for classification and measures how close the model's predicted probabilities for the correct class are. When an inappropriate loss function is used, it can cause the model to learn in a different way than what is needed for the problem.

C. A convolutional neural network processed an image by using filters, kernels. A kernel moves across regions of an image to produce a value for each position. These values form a feature map that can represent patterns like edges, textures, and shapes. The stride determines how much the kernel moves between calculations. Padding adds values (typically zeros) around the input so the kernel can process the edges and keep the feature maps dimensions. Pooling reduces the size of feature maps without discarding the important features. Fully connected layers connect every neuron in one layer to every neuron in the following layer so they use information from the previous layers to output a prediction. CNNs are effective for images because they can reuse kernels to find patterns which requires fewer parameters than a fully connected network.

Response:

Your explanation now covers nearly all parts of the original questions. The remaining gaps are mainly small wording issues, plus one missing concept in Part A.

A. Deep Neural Networks

Gaps and corrections

1. **Weights are multiplied by inputs, not added together by themselves.**

Your wording:

"These weights are then added together with the biases..."

More accurately, each input is multiplied by its weight. The weighted inputs are then added together with the bias:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

2. **The weighted sum is passed into the activation function.**

Instead of saying that values are "added to the activation function," say that the result is **passed into** the activation function.

3. **The bias explanation repeats "output."**

Your wording:

"The bias has the output shift its output."

A clearer explanation is:

The bias shifts the value passed into the activation function, allowing the neuron to activate even when the input values are zero.

4. **Forward propagation and backpropagation are still missing.**

If these have not been discussed in your class, it is reasonable to say so. However, they were part of the original question, so technically this is still a gap.

5. **Vanishing and exploding gradients are also missing.**

Again, you can identify these as topics your class has not covered rather than trying to explain them.

Corrected version

Weights determine how strongly each input affects a neuron. Each input is multiplied by its corresponding weight, and the weighted inputs are added together with a bias. The result is then passed into an activation function. The bias shifts the value passed into the activation function, while activation functions introduce nonlinear behavior that allows neural networks to learn complex patterns. Gradients determine the direction and amount by which the loss changes when weights and biases change. The learning rate controls the size of the steps

taken during gradient descent as the model attempts to minimize its loss. We have not yet discussed forward propagation, backpropagation, or vanishing and exploding gradients.

B. Loss Functions

Gaps and corrections

Your explanation is strong and answers nearly every part of the question.

1. **Minor grammar correction**

Change:

“what the models predicts”

To:

“what the model predicts”

2. **Cross-entropy can be explained more precisely.**

Cross-entropy does not only measure how close the correct-class probability is. It also **penalizes confident incorrect predictions more heavily** than uncertain predictions.

3. **You could distinguish common forms of cross-entropy.**

This is optional unless your class has covered it:

- **Binary cross-entropy:** used for binary classification
- **Categorical cross-entropy:** used for classification involving multiple classes

Corrected version

A loss function measures the difference between what the model predicts and the correct output. Mean squared error is commonly used for regression problems where the model predicts continuous values. It calculates the average squared difference between the predicted and correct values. Cross-entropy loss is commonly used for classification and measures how well the model’s predicted probabilities match the correct class. It gives a larger penalty when the model confidently predicts the wrong class. Using an inappropriate loss function can cause the model to learn in a way that does not match the goal of the problem.

C. Convolutional Neural Networks

Gaps and corrections

Your CNN explanation covers all the original requested concepts. Only a few details should be refined.

1. Grammar correction

Change:

"A convolutional neural network processed an image..."

To:

"A convolutional neural network processes an image..."

2. Clarify that filters and kernels mean the same thing in this context.

Change:

"using filters, kernels"

To:

"using filters, also called kernels"

3. Padding does not always preserve dimensions.

Padding **can** preserve the dimensions of the feature map, depending on the amount of padding, kernel size, and stride. It does not automatically preserve them.

4. Pooling does discard some information.

Your wording says pooling reduces size "without discarding the important features." Pooling does remove information, but it attempts to **retain the most useful or representative information**.

For example:

- Max pooling keeps the largest value in a region.
- Average pooling keeps the average value in a region.

5. Fully connected layers use extracted features, not all raw information.

The convolution and pooling layers extract features first. The fully connected layers then use those extracted features to make a prediction.

Corrected version

A convolutional neural network processes an image using filters, also called kernels. A kernel moves across small regions of an image and produces a value at each position. These values form a feature map that can represent patterns such as edges, textures, and shapes. Stride determines how far the kernel moves between calculations. Padding adds values, typically zeros, around the input so the kernel can process information near the edges. Padding can also help preserve the dimensions of the feature map. Pooling reduces the size of feature maps while attempting to retain the most important information. Fully connected layers connect every neuron in one layer to every neuron in the following layer. They use the features extracted by the previous layers to produce a final prediction. CNNs are effective for images because they reuse kernels to identify local patterns, requiring fewer parameters than a fully connected network.

Overall assessment

- **A:** Mostly correct, but revise the order of weights, weighted inputs, bias, and activation. Forward propagation, backpropagation, and gradient problems remain unaddressed.
- **B:** Complete aside from a minor grammar correction and a possible explanation of how cross-entropy penalizes confident errors.
- **C:** Complete and accurate overall. The main correction is that pooling does discard information, while attempting to keep the most useful features.

Main improvements

- **Accuracy:** Clarified the neuron calculation and the effects of padding and pooling.
- **Grammar:** Corrected verb agreement and several awkward phrases.
- **Precision:** Explained that fully connected layers use features extracted by earlier CNN layers.

5. I have learned from the exercise is how good Generative AI can be when wanting to improve your understanding of concepts. It can assess what you got wrong and explain how your thinking is wrong. It can answer your personalized questions as well. It can correct misunderstandings you have and can account for circumstances.

I would recommend to others when using GenAI is to be careful about what you mention and how you want to learn certain things. The GenAI assumed I already had

knowledge about certain things and gave me questions based on the misunderstanding. You have to be specific in your questions towards your AI that helps you the most.